Lab4 - Automatizando o deploy: CD com GitHub Actions e Azure Container Apps

IMD3005 - MLOPS - 2026.2 - prof. Adelson D. de Araujo

Onde paramos

No Lab3 você criou o `ci.yml`. A cada push ou pull request na `main`, ele testa o código e constrói a imagem Docker da API.

Tudo isso acontece numa máquina temporária que o GitHub cria para rodar o workflow, o runner. Quando o workflow termina, o GitHub apaga essa máquina, junto com a imagem e o container.

O problema

Ninguém fora dessa máquina temporária consegue usar a API. Ela funciona nos testes, mas não tem um endereço que outra pessoa possa acessar.

O objetivo deste lab

Colocar a API `intent-clf` numa URL pública na nuvem (Azure). Depois, fazer o GitHub atualizar essa URL sozinho sempre que você publicar uma nova versão do código.

O roteiro

1. Ajustar a API para ela dizer qual versão está rodando.
2. Guardar a imagem Docker num lugar da internet de onde a nuvem possa baixá-la.
3. Colocar a API no ar na nuvem, à mão, uma vez.
4. Escrever um segundo workflow no GitHub que repete esse caminho sozinho, com sua aprovação, e que permite voltar para uma versão anterior.

Antes de começar

- Seu repositório no GitHub tem o `ci.yml` do Lab3 rodando verde (jobs `test`, `docker-build` e `smoke`).
- Os secrets do W&B e do MongoDB Atlas que o `smoke` usa estão cadastrados no repositório.
- A API sobe localmente com o seu `.env`.

- Você tem Docker instalado.
- Você ativou a conta Azure for Students com o e-mail da UFRN (veja a aula).

Nos comandos, troque `SEU-USUARIO/SEU-REPO` pelo seu usuário e pelo nome do seu repositório no GitHub, sempre em minúsculas.

Parte 1 - Preparação

Fazer a API informar a versão

Mais adiante, você vai querer saber qual versão do código está no ar. A forma mais simples é perguntar à própria API.

A API vai ler a versão de uma variável de ambiente chamada `APP_VERSION`. Fazemos a mudança agora, antes de publicar qualquer versão, para que todas as imagens já tenham esse recurso.

1. Em `src/app.py`, ponha `import os` no topo do arquivo e troque a rota `GET /` por:

```python
import os  # no topo do arquivo

@app.get('/')
def root():
    return {'status': 'ok', 'version': os.getenv('APP_VERSION', 'dev')}
```

2. Rode a API localmente e abra `http://localhost:8000/`.
3. Se algum teste do Lab3 compara a resposta de `GET /` com `{'status': 'ok'}`, ajuste o teste.

Você deve ver: `{"status":"ok","version":"dev"}`. Aparece `dev` porque a variável `APP_VERSION` ainda não existe na sua máquina.

Deixar o ci.yml ser chamado por outro workflow

Na Parte 3, um segundo workflow vai rodar o `ci.yml` inteiro antes de publicar uma versão. Para isso, o `ci.yml` precisa aceitar ser chamado. É o gatilho `workflow_call`.

1. No `ci.yml`, acrescente `workflow_call:` ao bloco `on:`. Mantenha os gatilhos que você já tem; só a última linha é nova:

```yaml
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:
  workflow_call:      # linha nova
```

2. Faça commit das duas mudanças e envie:

```
git add src/app.py .github/workflows/ci.yml
git commit -m "GET / devolve version; ci.yml pode ser chamado"
git push
```

Você deve ver: o CI verde na aba Actions para esse commit. Só siga com o CI verde.

Parte 2 - Colocar a API no ar à mão

Aqui você faz uma vez, com as próprias mãos, o caminho que o workflow vai repetir depois. Isso também cria as peças que o workflow vai usar.

Guardar a imagem no GHCR

A nuvem não tem acesso à imagem que está no seu computador. Ela precisa baixá-la de um registro de imagens, um site que guarda imagens Docker com nome e etiqueta. Usaremos o GHCR (GitHub Container Registry), o registro do próprio GitHub.

1. Crie um token de acesso em GitHub > Settings > Developer settings > Personal access tokens > Tokens (classic), com o escopo `write:packages`. Esse token é a senha que o Docker usa para enviar ao GHCR.

2. Faça login, construa a imagem já com o endereço do GHCR e envie:

```
echo SEU_TOKEN | docker login ghcr.io -u SEU-USUARIO --password-stdin
docker build --platform linux/amd64 -t ghcr.io/SEU-USUARIO/SEU-REPO:latest .
docker push ghcr.io/SEU-USUARIO/SEU-REPO:latest
```

3. No seu perfil do GitHub, abra Packages e clique no pacote (é como o GitHub chama a imagem guardada). Em Package settings: - Change visibility > Public. Todo pacote nasce privado, e a nuvem precisa baixá-lo sem senha. Por isso nada secreto vai dentro da imagem; o `.env` fica fora. - Manage Actions access > Add repository > seu repositório, com papel Write. Assim o workflow do seu repositório poderá enviar imagens para este pacote.

Você deve ver: o pacote público em Packages, com a etiqueta `latest`.

Se der errado:

O que aparece	Por que acontece	O que fazer
`invalid reference format: repository name must be lowercase`	Maiúsculas no endereço	Escreva tudo em minúsculas

O que aparece	Por que acontece	O que fazer
Algo como `denied` ou `unauthorized` no `docker push`	Sem login ou token sem `write:packages`	Gere o token certo e refaça o login
Erro de plataforma ou build lento no Mac M1/M2	Imagem para `arm64`	Mantenha `--platform linux/amd64`

Dar um número de versão à imagem

Se toda imagem se chama `latest`, não dá para saber qual código está na nuvem, nem para qual voltar se algo quebrar. A solução é dar a cada versão um nome fixo, como `v1.0.0` (veja na aula como funciona a numeração MAJOR.MINOR.PATCH).

No git, esse nome fixo é uma tag: uma etiqueta presa a um commit, que nunca muda de lugar. Você cria a tag com `git tag` e a envia ao GitHub com `git push origin <nome-da-tag>`. Um `git push` comum não envia tags.

1. Marque o commit atual da `main` (o que tem a mudança no `app.py`) e envie a tag:

```
git switch main && git pull
git tag v1.0.0
git push origin v1.0.0
```

2. Dê à imagem a mesma etiqueta e envie ao GHCR:

```
docker tag ghcr.io/SEU-USUARIO/SEU-REPO:latest ghcr.io/SEU-USUARIO/SEU-REPO:v1.0.0
docker push ghcr.io/SEU-USUARIO/SEU-REPO:v1.0.0
```

Você deve ver: a etiqueta `v1.0.0` no pacote e a tag `v1.0.0` em Code > Tags do repositório.

Se der errado:

O que aparece	Por que acontece	O que fazer
A tag não aparece no GitHub	Usou `git push` comum	`git push origin v1.0.0`
`fatal: tag 'v1.0.0' already exists`	A tag já existe	Não recrie; use outro número

Criar o Container App

Precisamos de um computador na nuvem que rode a imagem e dê a ela uma URL pública. No Azure, o Container App faz isso: você informa o endereço da imagem, a porta e as variáveis de ambiente, e o Azure roda o container e cria a URL HTTPS. Não há máquina para administrar.

1. No portal do Azure, busque Container Apps > + Create.
2. Resource group (a pasta que agrupa seus recursos): crie `mlops-2026-2`. Nome do app: `intent-clf`. Escolha uma Region e deixe o Azure criar o Environment.
3. Aba Container: Image source Docker Hub or other registries, Image type Public, Registry login server `ghcr.io`, Image and tag `SEU-USUARIO/SEU-REPO:v1.0.0`.
4. CPU and memory: cerca de 2 vCPU e 4 GiB.
5. Environment variables: `WANDB_PROJECT`, `WANDB_MODELS`, `MONGO_DB` e `APP_VERSION` = `v1.0.0` (Source Manual); `WANDB_API_KEY` e `MONGO_URL` (Source Secret, o valor fica oculto).
6. Aba Ingress: Enabled, Accepting traffic from anywhere, Target port `8000`.
7. Review + create > Create > Go to resource.
8. Em Containers > Edit and deploy > Health probes, habilite a Startup probe: TCP, porta 8000, Period 10 s, Failure threshold 60. Assim o Azure espera até 10 minutos o app carregar os modelos antes de reiniciá-lo.
9. No MongoDB Atlas, em Network Access, libere `0.0.0.0/0`, para o Azure conseguir acessar o banco.

Você deve ver: no Overview do app, a Application Url (`https://intent-clf.<algo>.<região>.azurecontainerapps.io`). Abrindo essa URL, depois de alguns minutos: `{"status":"ok","version":"v1.0.0"}`. Guarde essa URL.

Se der errado:

O que aparece	Por que acontece	O que fazer
Erro de região ou de política (policy) ao criar	O plano Students só libera algumas regiões	Em Policy > Assignments > Allowed resource deployment regions, veja Allowed locations e escolha uma delas

O que aparece	Por que acontece	O que fazer
Revisão `Failed`, log com algo como `unauthorized` ou `manifest unknown`	Pacote privado ou etiqueta errada	Torne o pacote público; confira a etiqueta em Packages
App reiniciando, algo como `OOMKilled`	Pouca memória, startup lento ou porta errada	Mais CPU/memória, porta 8000, Startup probe; veja o Log stream
`GET /` funciona, `POST /predictions` dá 500	Atlas bloqueando o Azure ou `MONGO_URL` errado	Libere `0.0.0.0/0`; confira as variáveis

Dar ao GitHub permissão para mexer no Container App

Na Parte 3, o GitHub vai trocar a imagem do Container App sozinho. Para isso ele precisa entrar no Azure. Não queremos guardar uma senha no GitHub, então usamos o login sem senha OIDC: o Azure confia em pedidos que vêm de um job específico do seu repositório (veja a aula).

No Azure, quem recebe essa confiança é uma identidade gerenciada, uma espécie de usuário para programas. A credencial federada é a regra que diz qual repositório e qual job pode agir como essa identidade.

1. Busque Managed Identities > + Create, no resource group `mlops-2026-2` e na mesma região do app. Nome sugerido: `user-assigned-identity-1`.

2. Na identidade, abra Azure role assignments > Add role assignment: papel Contributor, escopo Resource group `mlops-2026-2`. Isso permite alterar recursos desse grupo.

3. Em Settings > Federated credentials > Add credential: - Scenario: GitHub Actions deploying Azure resources. - Organization: seu usuário do GitHub. O Organization ID é o campo `id` em `https://api.github.com/users/SEU-USUARIO`. - Repository: nome do repositório. O Repository ID é o campo `id` em `https://api.github.com/repos/SEU-USUARIO/SEU-REPO`. - Entity type: Environment. Nome: `production`. É o nome do ambiente de deploy que você vai criar no GitHub na Parte 3; só jobs ligados a ele poderão entrar.

4. No Overview da identidade, anote Client ID, Tenant ID e Subscription ID.

Você deve ver: a credencial em Federated credentials, com algo como `repo:SEU-USUARIO/SEU-REPO:environment:production`.

Se der errado:

O que aparece	Por que acontece	O que fazer
Entity type ficou Branch ou Tag	Opção errada	Apague e recrie com Environment = `production`
Contributor não aparece no resource group	Papel dado em outro escopo	Refaça com escopo `mlops-2026-2`

Parte 3 - Automatizar com o cd.yml

O `ci.yml` responde se o código funciona. O novo workflow, `cd.yml` (CD: entrega contínua), responde se a versão chegou à URL pública e funciona. Ele roda quando você envia uma tag `v...`.

O arquivo completo

Crie `.github/workflows/cd.yml` com o conteúdo abaixo e troque `SEU-USUARIO/SEU-REPO` nos dois lugares em que aparece. As seções seguintes explicam cada parte.

```yaml
# .github/workflows/cd.yml
name: CD

on:
  push:
    tags: ['v*']              # versão nova: git push origin v1.0.1
  workflow_dispatch:          # rollback: botão "Run workflow"
    inputs:
      tag:
        description: 'Tag já publicada no GHCR (ex.: v1.0.0)'
        required: true

env:
  TAG: ${{ inputs.tag || github.ref_name }}   # tag digitada ou tag enviada
  APP_NAME: intent-clf
  RESOURCE_GROUP: mlops-2026-2

jobs:
  # 1. Roda o ci.yml inteiro (test + smoke) no commit da tag.
  #    Só segue se tudo passar. No rollback, é pulado.
  ci:
    if: github.event_name == 'push'
    uses: ./.github/workflows/ci.yml
    secrets: inherit          # o smoke precisa dos segredos do W&B e do Mongo

  # 2. Constrói a imagem e publica no GHCR com a tag da versão
  release:
    needs: ci
```

```yaml
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - name: Build e push com a versão
        run: |
          IMAGE=ghcr.io/SEU-USUARIO/SEU-REPO   # em minúsculas
          docker build -t "$IMAGE:$TAG" .
          docker push "$IMAGE:$TAG"

  # 3. Troca a imagem no Azure Container Apps (espera aprovação)
  deploy:
    needs: release
    if: ${{ !failure() && !cancelled() }}   # roda também no rollback (release pulado)
    environment: production
    runs-on: ubuntu-latest
    permissions:
      id-token: write         # pedir o "crachá" OIDC
      contents: read
    steps:
      - name: Login no Azure via OIDC
        uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
      - name: Atualizar imagem e versão no Container Apps
        run: |
          IMAGE=ghcr.io/SEU-USUARIO/SEU-REPO   # em minúsculas
          az containerapp update --name "$APP_NAME" --resource-group "$RESOURCE_GROUP" \
            --image "$IMAGE:$TAG" \
            --set-env-vars APP_VERSION="$TAG"

  # 4. Testa a URL pública e confere se a versão certa está no ar
  verify:
    needs: deploy
    if: ${{ !failure() && !cancelled() }}
    runs-on: ubuntu-latest
    steps:
      - run: |
          URL=${{ vars.APP_URL }}
          MSG='{"nome":"cd","text":"não entendi nada"}'
          for _ in $(seq 90); do            # até 15 min (startup lento)
            if curl -fsS "$URL/" | grep -q "\"$TAG\"" &&
               curl -fsS -X POST "$URL/predictions" \
                 -H "Content-Type: application/json" -d "$MSG"; then
              echo "OK: $TAG no ar"; exit 0
            fi
            sleep 10
          done
```

```
          echo "Falhou: $TAG não respondeu. Faça o rollback."; exit 1
```

Gatilhos e variáveis

```yaml
on:
  push:
    tags: ['v*']
  workflow_dispatch:
    inputs:
      tag: ...
env:
  TAG: ${{ inputs.tag || github.ref_name }}
```

O workflow roda em dois casos: quando chega ao GitHub uma tag que começa com `v`, ou quando você clica no botão Run workflow na aba Actions (gatilho `workflow_dispatch`). Um push comum na `main` não dispara o CD; só o CI.

`TAG` guarda a versão com que todos os jobs trabalham. Num push de tag, `github.ref_name` é o nome da tag. No botão, vale a tag digitada; por isso `inputs.tag` vem primeiro. `APP_NAME` e `RESOURCE_GROUP` precisam ser iguais aos nomes que você criou no Azure.

Job ci

```yaml
  ci:
    if: github.event_name == 'push'
    uses: ./.github/workflows/ci.yml
    secrets: inherit
```

Antes de publicar, roda o seu `ci.yml` inteiro no commit da tag. Se algum teste falhar, nada é publicado. `secrets: inherit` repassa os secrets do repositório, que o `smoke` usa. O `if` faz o job ser pulado quando você usa o botão, porque aí não há código novo para testar.

Se o job ci falhar:

O que aparece	Por que acontece	O que fazer
Falha sem jobs, com algo como `workflow is not reusable as it is missing a on.workflow_call trigger`	`ci.yml` sem `workflow_call`, ou tag num commit anterior a essa mudança	Volte à Parte 1; commit e tag nova
O smoke reclama de segredo vazio	Faltou `secrets: inherit`	Acrescente a linha

Job release

```
  release:
    needs: ci
    permissions:
      packages: write
    steps: ... docker build -t "$IMAGE:$TAG" . && docker push "$IMAGE:$TAG"
```

Faz sozinho o que você fez na Parte 2: login no GHCR, build e push da imagem com a etiqueta igual à tag. `needs: ci` faz ele esperar o CI passar. `packages: write` dá permissão de enviar ao GHCR, e o login usa o token que o GitHub cria para cada execução (`GITHUB_TOKEN`), sem você cadastrar nada.

Se o job release falhar:

O que aparece	Por que acontece	O que fazer
`denied: permission_denied: write_package`	Repositório sem acesso Write ao pacote	Package settings > Manage Actions access > seu repo como Write
`invalid reference format: repository name must be lowercase`	Maiúsculas em `IMAGE`	Endereço em minúsculas

Job deploy

```
  deploy:
    needs: release
    environment: production
    permissions:
      id-token: write
    steps: azure/login ... az containerapp update --image "$IMAGE:$TAG" --set-env-vars
APP_VERSION="$TAG"
```

Entra no Azure pelo OIDC e troca a imagem do Container App pela nova versão. Também grava a tag em `APP_VERSION`, que a API devolve em `GET /`. `id-token: write` permite ao job pedir o "crachá" OIDC. `environment: production` liga o job ao ambiente de deploy: é isso que a credencial federada exige, e é aí que fica a regra de aprovação.

O Azure sobe a versão nova ao lado da antiga e só passa o tráfego quando a nova fica saudável. Cada versão aparece como uma revisão em Revisions and replicas.

Se o job deploy falhar:

O que aparece	Por que acontece	O que fazer
Algo como `Login failed ... Not all values are present`	Secrets `AZURE_*` ausentes ou com outro nome	Cadastre os três com o nome exato
`AADSTS70021: No matching federated identity record found`, subject com `ref:refs/tags/...`	Job sem `environment: production`	Acrescente a linha
Mesmo erro, subject com `environment:production`	Credencial com usuário, repo ou nome diferente (ex. `Production`)	Recrie a credencial copiando o subject do log
Algo como `Unable to get ACTIONS_ID_TOKEN_REQUEST_URL`	Faltou `id-token: write`	Acrescente em `permissions`
`(AuthorizationFailed) ... Microsoft.App/containerApps/write`	Identidade sem Contributor no resource group	Refaça a atribuição em `mlops-2026-2`
Algo como `ResourceNotFound`	`APP_NAME` ou `RESOURCE_GROUP` diferentes	Corrija o bloco `env`
Algo como `Tag "v1.0.1" is not allowed to deploy to production`	Ambiente restrito a branches	Em Deployment branches and tags, No restriction

Job verify

```
verify:
  needs: deploy
  steps: curl "$URL/" | grep "\"$TAG\"" && curl -X POST "$URL/predictions" ...
```

O deploy pode ficar verde mesmo se a revisão nova falhar, porque a antiga continua respondendo. O verify confere na URL pública se `GET /` mostra a tag certa e se uma predição funciona. Tenta a cada 10 segundos, por até 15 minutos, porque o app demora a carregar os modelos. A URL vem da variável `APP_URL`.

Se o job verify falhar:

O que aparece	Por que acontece	O que fazer
`curl: (3) URL rejected: No host part in the URL` repetido	`APP_URL` não existe ou foi criada como secret	Crie em Variables
Vermelho após 15 minutos, sem erro de curl	`APP_URL` sem `https://`	Corrija o valor
`GET /` mostra só `{"status":"ok"}`	Imagem sem a mudança do `app.py`	Commit do `app.py` e tag nova
`GET /` mostra a versão anterior	Revisão nova falhou	Revisions e Log stream no portal
`curl: (22) ... error: 500` no POST	Atlas bloqueando ou `MONGO_URL` errado	`0.0.0.0/0` no Atlas; variáveis do app
`curl: (22) ... error: 404`	URL errada	Copie a Application Url do Overview

Botão de rollback

```yaml
on:
  workflow_dispatch:
    inputs:
      tag: ...
jobs:
  ci:
    if: github.event_name == 'push'
    ...
  release:
    if: github.event_name == 'push'
    ...
  deploy:
    if: ${{ !failure() && !cancelled() }}
    ...
  verify:
    if: ${{ !failure() && !cancelled() }}
    ...
```

Rollback é voltar para uma versão anterior. Aqui isso significa reimplantar uma imagem que já está no GHCR, sem construir nada. No botão, `ci` e `release` são pulados.

Normalmente, um job é pulado quando um job de que ele depende (`needs`) foi pulado. A condição `!failure() && !cancelled()` deixa o deploy e o verify rodarem mesmo assim. Num push de tag, ela continua barrando o deploy se o CI ou o release falharem.

Se o rollback falhar:

O que aparece	Por que acontece	O que fazer
Botão Run workflow não aparece	`cd.yml` não está na `main`	Commit e push na `main`
`deploy` e `verify` pulados	Faltou o `if` com `!failure()`	Acrescente nos dois
Revisão falha com algo como `manifest unknown`	Tag digitada não existe (ex. `1.0.1` sem `v`)	Confira em Packages

Configurar o GitHub

O `cd.yml` usa informações que não podem ficar no arquivo: os códigos da identidade, a URL do app e a regra de aprovação.

1. Secrets: Settings > Secrets and variables > Actions > New repository secret. Crie `AZURE_CLIENT_ID` (o Client ID da identidade, não o Object ID), `AZURE_TENANT_ID` e `AZURE_SUBSCRIPTION_ID`.
2. Variável: na mesma tela, aba Variables > New repository variable. Nome `APP_URL`, valor a Application Url, com `https://` e sem barra no fim. Variável, e não secret, porque a URL é pública.
3. Environment: Settings > Environments > New environment > `production`. Um environment é um destino de deploy com regras. Este não tem relação com o Environment do Container Apps no Azure; só compartilham a palavra.
4. No environment `production`, marque Required reviewers, adicione você e clique em Save protection rules. Assim o deploy espera alguém aprovar. Em Deployment branches and tags, deixe No restriction.

Você deve ver: três secrets, uma variável e o environment `production` com você como revisor.

Se der errado:

O que aparece	Por que acontece	O que fazer
Opção Required reviewers indisponível	Repositório privado no plano grátis	Torne o repositório público

Testar: publicar a v1.0.1

Agora você publica uma versão nova sem tocar no Azure.

1. Faça commit do `cd.yml` e envie. Espere o CI ficar verde.

```
git add .github/workflows/cd.yml
git commit -m "cd.yml"
git push
```

2. Crie a tag no commit novo e envie:

```
git tag v1.0.1
git push origin v1.0.1
```

3. Na aba Actions, abra o run CD chamado `v1.0.1` e acompanhe: - `ci`: aparece como um bloco com os jobs do `ci.yml`. `test` e `smoke` ficam verdes; `docker-build` aparece pulado. - `release`: verde. Em Packages surge a etiqueta `v1.0.1`. - `deploy`: fica amarelo, em Waiting. Clique em Review deployments, marque `production` e Approve and deploy. Tire o print antes de aprovar. - `verify`: depois de alguns minutos, o log termina com `OK: v1.0.1 no ar`.

4. Abra a Application Url no navegador.

Você deve ver: os quatro jobs verdes e `{"status":"ok","version":"v1.0.1"}`. No Azure, em Revisions and replicas, uma revisão com a imagem `:v1.0.1`.

Uma tag congela o conteúdo do commit dela, inclusive o `cd.yml`. Se corrigir o arquivo, faça commit e crie uma tag nova no commit novo. Nomes de tag não podem ser reusados.

Se der errado:

O que aparece	Por que acontece	O que fazer
Nenhum run CD aparece	Tag não enviada, nome sem v, ou arquivo fora de `.github/workflows/`	`git push origin v1.0.1`; confira com `git ls-remote --tags origin`
Run falha em segundos, algo como	Erro de indentação no YAML	Leia a anotação no run

O que aparece	Por que acontece	O que fazer
`The workflow is not valid`		
`deploy` roda sem pedir aprovação	Regra não salva	Salve Required reviewers
Botão de aprovar não aparece	Você não é revisor, ou Prevent self-review ligado	Adicione-se; desligue a opção
`fatal: tag 'v1.0.1' already exists`	Reuso de nome	Use `v1.0.2`

Exercício: publicar a v1.0.2 e voltar para a v1.0.1

Imagine que a v1.0.2 saiu com problema. Você vai voltar para a v1.0.1 sem construir nada.

1. Publique a v1.0.2. Não é preciso commit novo: uma tag pode apontar para o mesmo commit de outra.

```
git tag v1.0.2
git push origin v1.0.2
```

2. Aprove o deploy e espere o verify verde. A URL deve mostrar `"version":"v1.0.2"`.
3. Faça o rollback: Actions > CD > Run workflow. Branch `main`, tag `v1.0.1`, Run workflow.
4. Aprove o deploy.

Você deve ver: `ci` e `release` cinza (pulados), `deploy` aguardando aprovação, `verify` verde com `OK: v1.0.1 no ar`, e a URL mostrando `"version":"v1.0.1"`. O rollback só trocou a imagem.

Parte 4 - Entrega

Prazo: 13/11/2026.

1. Link do repositório público com `.github/workflows/cd.yml`.
2. PDF com prints de: - Overview do Container App, com a Application Url e a imagem com a tag; - job `deploy` aguardando aprovação (v1.0.1); - run da v1.0.1 com `release`, `deploy` e `verify` verdes; - log do `verify` com `OK: v1.0.1 no ar`; - `GET /` mostrando `"version":"v1.0.2"` e, depois do Run workflow, `"version":"v1.0.1"`; - run do rollback com `ci` e `release` pulados; - resource group `mlops-2026-2` apagado.

Ao terminar e tirar os prints, apague o resource group inteiro para parar o consumo dos créditos.

Nunca faça commit do `.env` nem de tokens.